

SHIELD

Outcome Policy Infrastructure for Autonomous On-Chain Agents

Technical & Product Whitepaper

Version 1.0 | October 2026

Status: Research / Pre-MVP

Token: None | Custody: Non-custodial

Focus: Ethereum-compatible smart accounts and autonomous agents

This document is a conceptual research and product whitepaper. It is not financial, investment, legal, or security advice, and it is not an offer of any token or financial product. Technical claims about third-party standards must be verified against primary sources before any public use.

Contents

1. Abstract 3

2. The Problem 3

3. Existing Solutions 3

4. Core Thesis: Three Security Layers 4

5. Design Principles 4

6. Architecture 5

7. Policy Model 5

8. On-Chain Enforcement in the MVP 6

9. Outcome Enforcement: Research Track 6

10. Simulation 7

11. Contract Risk Registry 7

12. Intent and the Role of AI 8

13. Threat Model and Failure Modes 8

14. Ethereum Transaction Assertions 8

15. Competitive Positioning 9

16. MVP Scope 9

17. Roadmap 10

18. Business Model 10

19. Token Position 11

20. Optional Future: Protection Layer 11

21. Validation Plan 11

22. Legal and Regulatory Considerations 11

23. Open Questions and Risks 11

24. References (to be verified before publication) 11

1. Abstract

AI agents can increasingly trade, rebalance, pay, and operate DeFi protocols without a human signing each action. This changes the security question from "is this transaction authorized?" to "is this transaction still consistent with the economic constraints under which the agent was authorized?"

Smart accounts, session keys, modules, and validators already answer part of this: they limit **what an agent may call** (contracts, tokens, spend, time, rate). SHIELD does not replace them. SHIELD investigates a complementary layer: constraining **what the resulting transaction may change**.

	Question	Example
Permission	What may the agent call?	May spend up to 5,000 USDC on Uniswap for 24 hours
Outcome policy	What may the transaction produce?	Must receive at least a minimum WBTC, create no new approval, touch no unapproved contract

The MVP is deliberately conservative. It does **not** claim to understand arbitrary natural-language intent, to predict all economic consequences, or to guarantee that a contract is safe. It delivers: (1) deterministic policy primitives, (2) transaction simulation, (3) contract-risk checks, (4) at least one genuine on-chain enforcement mechanism, and (5) smart-account integration. General outcome introspection is treated as a separate research track.

2. The Problem

2.1 From human-signed to agent-executed transactions

Human wallet:	Human -> Wallet -> Review -> Sign -> Blockchain
Agent wallet:	Human -> Agent -> Many transactions -> Blockchain

A human authorizes an agent once; the agent may then act for minutes, days, or indefinitely. Security can no longer rely on a person inspecting each transaction.

2.2 Why a valid signature is not enough

A signature authorizes a transaction request; the economic result depends on the code and state encountered at execution. Consider an agent that calls an approved DEX and spends only 4,900 USDC (within its spending limit) but receives assets worth 2,000 USD because of poor liquidity, a sandwich attack, or a malicious route. The permission was respected; the user's objective was not.

- Changing market state and price movement
- MEV and sandwich attacks
- Proxy implementation changes after approval
- Malicious or adaptive routing and contract behaviour
- Unintended approvals or transfers bundled in a call

3. Existing Solutions

SHIELD does not claim that agent permissions are an empty market. They are not.

- **ERC-4337** - account abstraction with customizable validation logic; foundation for session keys, validators, and delegated execution.

- **Safe** - modular smart accounts with modules and guards, including post-execution checks.
- **ZeroDev and similar session-key systems** - permissions limiting contracts, tokens, gas, rate, and time.
- **Transaction simulation and contract-security tools** - predict effects and flag risky contracts.
- **Native transaction assertions (research)** - a possible future protocol primitive for outcome enforcement (Section 14).

Therefore, "AI agent + wallet + spending limit" is not a sufficient reason for SHIELD to exist. SHIELD's thesis is narrower and must be validated: the developer experience that combines authorization, simulation, contract intelligence, and deterministic outcome policies for autonomous agents is not yet standardized.

4. Core Thesis: Three Security Layers

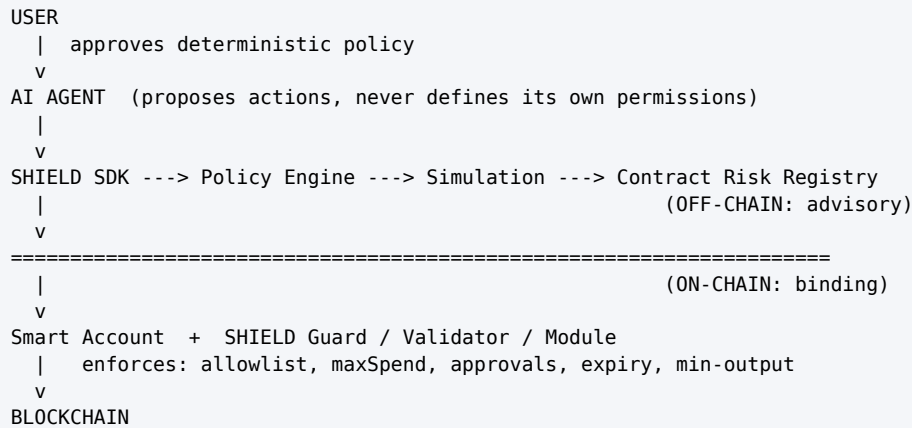
Layer	Purpose	Examples	Maturity
1. Authorization	May the agent execute at all?	Allowed contract/token, max spend, expiry, rate limit, session key	Well served by existing infrastructure
2. Pre-execution analysis	What is the transaction expected to do?	Decoding, simulation, contract analysis, risk checks	Advisory; can be stale or fooled
3. Outcome enforcement	What is the transaction permitted to produce?	Min received, forbidden approval, allowed state changes	Partially feasible today; general solution is open research

Long-term pipeline: Authorization -> Pre-execution analysis -> Outcome policy -> On-chain enforcement -> Blockchain.

5. Design Principles

- **Non-custodial.** SHIELD never holds user funds.
- **Less authority than the agent it protects.** A compromised SHIELD service must not be able to move user funds.
- **The binding boundary is on-chain.** Off-chain components advise; smart-account contracts enforce.
- **AI proposes, never decides.** Models may draft policies; users approve them; contracts enforce them.
- **Defense in depth.** No single component is treated as sufficient.
- **Honest claims.** Every feature is labelled Advisory, Policy, or Enforcement, and nothing is marketed as Enforcement until proven.
- **Deterministic first.** Intent compilation comes only after deterministic primitives are reliable.

6. Architecture



The server may fail, be unavailable, or be compromised. The fundamental permission boundary must remain on-chain.

6.1 Components

Component	Role
SHIELD SDK	Developer API: policy(), simulate(), check(), compile()
Policy Engine	Turns structured rules into deterministic, machine-checkable policy objects
Simulation Engine	Runs the transaction on current or forked state and extracts expected effects
Contract Risk Registry	Security metadata and Allow / Warn / Block classification
Enforcement Adapters	On-chain modules for Safe, ERC-4337 accounts, later EIP-7702 and others

7. Policy Model

The MVP policy language is intentionally small and fully deterministic.

```

{
  "chain": "ethereum",
  "allowedContracts": ["UNISWAP_ROUTER"],
  "allowedTokens": ["USDC", "WBTC"],
  "maxSpend": { "USDC": "5000" },
  "maxSlippageBps": 200,
  "forbid": ["newApproval", "unknownRecipient"],
  "session": { "durationSec": 86400, "rateLimitPerHour": 10 }
}

```

Primitive	Enforcement today	Note
maxSpend, allowedContract, allowedToken, allowedRecipient	On-chain (validator / guard)	Core MVP
deadline, sessionDuration, rateLimit	On-chain	Core MVP
forbiddenApproval	On-chain, with caveats	Must cover approve, increaseAllowance, permit, Permit2, setApprovalForAll
protocolNativeSlippage	On-chain, per router	Validator decodes calldata and enforces a policy-derived minimum
minimumReceived (generic)	Research	Needs safe post-execution state inspection

Primitive	Enforcement today	Note
maxDrawdown, portfolio intent	Out of scope	Requires valuation, oracle, and multi-transaction semantics

8. On-Chain Enforcement in the MVP

A design in which a SHIELD server returns PASS and the transaction then proceeds unchecked is not acceptable: anyone who compromises or bypasses the server bypasses the policy. The MVP therefore requires at least one binding on-chain mechanism.

8.1 Safe-based path

Agent -> SHIELD SDK -> Safe (Module / Guard) -> Ethereum

A guard or module enforces target allowlists, token amounts, and approvals independently of any server. Safe guards can run post-execution checks, but a blocking guard that is broken can freeze an account (denial of service). The first enforcement contract must be treated as security-critical code: minimal, formally reviewed, and independently audited.

8.2 ERC-4337 path

Agent -> UserOperation -> SHIELD Validator -> EntryPoint -> Smart Account

A validator checks session key, allowed contract, maximum token amount, and expiry during validation. Bundler and paymaster rules constrain what validation logic may read, which must be respected in the design.

8.3 Bypass test

Minimum trust property: if the agent ignores the SHIELD server and submits a 2,000 USDC transfer to an unknown contract directly, the smart-account validator must still reject it because the amount exceeds the 500 USDC limit and the target is not allowlisted.

8.4 Policy and registry governance

If an allowlist or registry on-chain can be changed by SHIELD, SHIELD becomes a trusted party. The owner of the wallet must control policy changes. Any SHIELD-maintained registry data must be consumed through a timelock, be optional, and never be able to widen permissions without owner approval.

9. Outcome Enforcement: Research Track

The central technical question: how can outcomes be enforced without a native full state-diff primitive? Four approaches are considered.

Approach	Description	Strengths	Limits
A. Balance before/after	Guard reads ERC-20 balances before and after execution and checks a minimum	Simple, generic for tokens	Reentrancy, mid-transaction balance manipulation, rebasing and fee-on-transfer tokens; audit-heavy
B. Safe post-execution guard	Guard inspects selected state after execution	Fits existing accounts	Sees only state it can read; DoS risk if flawed

Approach	Description	Strengths	Limits
C. Protocol-native slippage	Use amountOutMinimum or equivalent	Enforced by the DEX itself	Agent may set it to 0 unless the validator decodes calldata and enforces a policy minimum; per-router support
D. Native assertions	Compile policy to a protocol-level post-transaction assertion	Strongest, general	Not deployed; proposal stage

Important: amountOutMinimum is only a safeguard if the on-chain validator verifies that the value in calldata is at least the policy-derived minimum. Otherwise the agent can simply set it to zero. Support is therefore router-by-router, not generic.

10. Simulation

Simulation predicts behaviour under a particular state. It is not proof, and it is not the security boundary.

10.1 Freshness

Each simulation carries a timestamp and block reference. A configurable maximum age (per chain and use case) marks results STALE, forcing re-simulation. Example: simulated at 12:00:00, checked at 12:00:18, maximum age 10 seconds: result STALE.

10.2 Re-simulation

```
Simulate -> Sign -> Pre-submission check -> Re-simulate -> Submit
(calldata changed => FAIL; state moved beyond tolerance => RE-SIMULATE)
```

This reduces, but does not eliminate, staleness risk. On-chain constraints remain the safety net.

10.3 Adaptive contracts

A contract can behave differently depending on caller, block state, oracle state, liquidity, ordering, or implementation. SHIELD therefore never claims that a simulator proves a contract safe.

11. Contract Risk Registry

A data layer of security signals per contract: verified source, proxy and upgradeability status, admin addresses, implementation history, audits, deployment age, protocol classification, historical exploits, known malicious behaviour. Output: Allow, Warn, or Block.

- Known Uniswap router: Allow. Newly deployed unknown contract: Warn. Known malicious contract: Block.
- A new implementation behind an approved proxy triggers re-evaluation.
- The registry is a risk signal, never a safety guarantee, and scores must not be presented as absolute.

12. Intent and the Role of AI

Natural-language intent such as "keep my portfolio safe and lose no more than 5%" is not deterministic: portfolio definition, valuation source, time window, realized vs unrealized loss, leverage, liabilities, and cross-chain positions are all unspecified. SHIELD does not claim arbitrary intent verification in version 1.

Human intent -> AI proposes -> Policy compiler -> Deterministic policy
-> User reviews and approves -> On-chain enforcement

AI is a policy-authoring assistant. It is never the enforcement layer and never the authority that defines its own permissions. Multi-transaction intents (for example rebalancing to 60/40) require evaluating the final portfolio state across a workflow and are outside the MVP.

13. Threat Model and Failure Modes

Threat	Description	Mitigation / status
SHIELD server compromise	Attacker forges PASS	On-chain enforcement; server has less authority than the agent
Approval bypass	permit, Permit2, setApprovalForAll evade approve checks	Explicit vector list; calldata decoding; tests
Stale or fooled simulation	State changes, adaptive contracts	Freshness limits, re-simulation, on-chain bounds
Registry poisoning	Malicious allowlist update	Owner control, timelock, no silent widening
Oracle manipulation / MEV	Wrong price or reordering	Protocol-native bounds; not fully solvable
Approved protocol exploited	Policy allows a compromised protocol	Registry monitoring; optional future protection layer
Proxy upgrade	Implementation changes after approval	Implementation pinning or re-approval
Guard bug / DoS	Broken guard freezes account	Minimal code, audits, owner-controlled escape hatch
Bad policy or bad strategy	User or agent makes a bad decision within policy	Not preventable; reduces blast radius only

SHIELD is not a guarantee of profitability or absolute safety. It reduces the set of actions and outcomes an autonomous agent can produce.

14. Ethereum Transaction Assertions

Protocol-level research on native transaction assertions explores letting a transaction be followed by an assertion that inspects net state changes and events and reverts execution if a condition fails. EIP-7906 ("Transaction Assertions via State Diff Opcode") is one proposal, described as a Draft Standards Track: Core proposal that adds state-diff opcodes and a post-transaction frame, and that depends on the frame-transaction proposal EIP-8141.

- EIP-7906 is **not** a deployed Ethereum primitive, and SHIELD does not depend on it.
- SHIELD must never claim to "use Ethereum transaction assertions" in the MVP.

- Position: SHIELD policies are designed so they could later compile into native assertions, making SHIELD a policy abstraction layer rather than something made obsolete.

Verification required: dates, titles, opcode names, and dependencies cited here (including the Ethereum Foundation article dated 5 October 2026) were taken from source notes and must be checked against the primary sources listed in Section 24 before external publication.

15. Competitive Positioning

System	Main capability	Relationship to SHIELD
ERC-4337	Account abstraction, custom validation	Foundation
Safe (modules, guards)	Modular accounts, custom enforcement	Integration target; partial overlap
ZeroDev and session-key systems	Permissions: contract, token, gas, rate, time	Authorization layer; overlap on basic limits
Simulation tools	Predict transaction effects	Input signal, not enforcement
Contract security tools	Detect risky contracts	Data source or competitor on registry
Native assertions (research)	Potential protocol-level outcome enforcement	Possible compile target
SHIELD	Policy + simulation + risk + outcome enforcement for agents	Developer experience and outcome focus

The differentiation is a hypothesis, not a fact. Candidate sources of value to test: a simpler policy DX for agent developers, router-aware outcome checks, and the contract risk registry as a data product.

16. MVP Scope

16.1 In scope (v0.1)

- Policy engine: maxSpend, allowedContract, allowedToken, allowedRecipient, deadline, rateLimit, sessionDuration, forbiddenApproval, protocolNativeSlippage.
- Transaction decoding and simulation with freshness checks.
- Contract registry v0: allowlist, blocklist, verified-protocol list, proxy detection.
- One on-chain enforcement path (Safe guard or ERC-4337 validator) on a testnet, with one router.
- Dashboard: balances, transactions, blocked transactions, policy violations.

16.2 Explicitly not in scope

- Portfolio drawdown limits and general intent verification.
- Generic minimum-received enforcement for arbitrary protocols.
- Insurance pools, underwriters, tokens, DAO governance.
- Custody of user funds.

16.3 Developer example

```
const policy = shield.policy({
  allowedContracts: [UNISWAP_ROUTER],
  maxSpend: { USDC: 5000 },
  maxSlippageBps: 200,
  forbid: ['newApproval']
});
const result = await shield.check(policy, tx); // PASS | WARN | FAIL
// Binding enforcement is performed by the on-chain module, not by this call.
```

16.4 Success criteria

- Demo: an agent swap on testnet where an out-of-policy transaction submitted directly (bypassing the server) is rejected on-chain.
- Router-aware minimum-output enforcement cannot be set to zero by the agent.
- Approval-vector test suite passes (approve, increaseAllowance, permit, Permit2, setApprovalForAll).
- External review of the enforcement contract before any mainnet use.

17. Roadmap

Phase	Focus	Exit signal
0. Validation	10-20 developer interviews; define target agent frameworks	Clear, repeated pain point; willingness to pay
1. MVP v0.1	Policy engine, simulation, one on-chain guard/validator, testnet demo	Bypass test passes; 3-5 pilot integrations
2. Hardening	Audit, second account type, more routers, registry v1 with timelock	Audit complete; mainnet limited beta
3. Outcome research	Post-execution checks (Approach A/B) with formal analysis	Safe design for selected outcome policies
4. Intent compiler	AI-assisted policy authoring with user review	Reviewable, deterministic output
5. Optional protection layer	Integrate third-party protection providers	Real demand; legal review done

18. Business Model

SHIELD is software infrastructure. Initial revenue is expected from developer tooling, not insurance premiums.

Tier	Offer
Free	Core SDK, limited simulations, community registry
Pro	Higher usage, monitoring, dashboard, alerts
API (usage-based)	Simulation and risk-check calls at volume
Enterprise	Custom policies, SLAs, private registry, integration support

Pricing is a placeholder and must be validated with real customers.

19. Token Position

SHIELD has no token and no plan to issue one in the MVP. A token is considered only if a network function cannot be done well without it, for example decentralized validators or attestors who verify policy compliance and must stake and be slashed for dishonesty. If such a network ever exists, protection payouts should not depend on the native token, which could lose value exactly when capital is needed. The token must not exist merely to create a tradable asset.

20. Optional Future: Protection Layer

Insurance-style protection is not the product. If demand appears after agents use SHIELD, protection can be offered by integrating existing providers behind SHIELD, so SHIELD acts as a routing and policy layer rather than an underwriter. This adds regulatory, pricing, adverse-selection, and correlation issues that are out of scope until real loss data exists.

21. Validation Plan

- Interview 10-20 developers building agents that hold real funds. Ask how they limit agents today, what has gone wrong, and what they would pay for.
- Build the narrow demo in Section 16 and have developers try to break it.
- Run 3-5 pilot integrations and measure blocked transactions, false positives, and integration time.
- Decide go / pivot / stop at defined checkpoints, not by sunk cost.

22. Legal and Regulatory Considerations

A non-custodial software tool differs from custody, trading, or insurance, but classification depends on jurisdiction and exact functionality. Relevant areas include financial-services rules, software liability, data protection, and sanctions compliance. Any future token or protection product raises additional securities, commodities, or insurance questions. Obtain qualified legal advice before launch. This document is not legal advice.

23. Open Questions and Risks

- Is the outcome-focused developer experience valuable enough versus session-key tools that will keep adding features?
- Can router-aware outcome checks be built without excessive per-protocol maintenance?
- How large is the market of agents holding meaningful real funds today?
- How will the registry stay current and trustworthy without becoming a central authority?
- What liability exists if a policy fails to prevent a loss?
- Will native assertions arrive, and if so, how does SHIELD stay relevant on top of them?

24. References (to be verified before publication)

- ERC-4337: Account Abstraction Using Alt Mempool (eips.ethereum.org).

- EIP-7906: Transaction Assertions via State Diff Opcode (eips.ethereum.org); verify status, dates, and dependencies.
- EIP-8141: frame transactions (eips.ethereum.org); verify status.
- Ethereum Foundation blog: article on native transaction assertions (check exact title, date, and wording at blog.ethereum.org).
- Safe documentation: modules and guards (docs.safe.global), including guard risk warnings.
- ZeroDev documentation: permissions and session keys (docs.zerodev.app).
- Uniswap documentation: router and slippage parameters (docs.uniswap.org).